architecture

Architecture

Preston-Check is a bash-based pre-deployment security scanner with an open-core commercial model. This document describes the runtime architecture, the open-source / commercial split, the component layout, and the data flow during a scan.

Component overview

The scanner has five layers, four of them open-source.

The **engine** is `preston-check.sh` plus the `lib/` modules. It loads the license, detects the source language, parses check metadata, filters by tier, runs the selected checks, aggregates results, and renders the report. All of this runs locally on the user's machine; no source code ever leaves the host without an explicit opt-in flag.

The **check catalog** is the collection of bash scripts under `checks/`. Each script is sourced into the runner's shell, and exposes its findings via the `record` function the runner exports. Checks live in one of six locations: the legacy `checks/` root (treated as core for backward compatibility), `checks/core/` (canonical maintainer-authored checks), and the community trust-tier directories `checks/community/{verified, accepted,proposed}/`. As of v1.7.3 the catalog totals 294 checks: 284 in the main suite plus the 10 deep checks in the smart-contract module.

The **smart-contract audit module** at `modules/smart-contract-audit/` is a separate runner for deep contract audits. It reuses the engine's `record` function and metadata parser but ships its own `audit.sh` entrypoint, its own check directory, and three integration wrappers (Slither, Mythril, Echidna) that opt-in via flags. Runtime profile is explicitly different from the main scanner — minutes per contract, not seconds per repo.

The **AI augmentation libs** at `lib/ai_analyze.sh` and `lib/ai_autofix.sh` attach to the engine via `--ai-augment` and `--ai-fix`. They speak Anthropic, OpenAI, or local Ollama, cache responses per-finding under `~/.preston-check/ai-cache/`, and attach analysis + suggested patches to the report addendum next to the file:line:content findings. Both are strict no-ops under `--airgap` and graceful no-ops without API keys configured.

The **commercial layer** is a separate proprietary product not included in this repository. It receives generated reports from Pro/Enterprise customers and produces the auditor-ready packaging: compliance evidence bundling, branded PDF generation, multi-repo dashboards, customer portal, license issuance backend, and SSO integration. Designed in `docs/portals-and-kpis.md`.

Distribution surface

The same engine ships through five channels, each of which is exercised on every release tag via `.github/workflows/release.yml`:

- **Source tarball** — `preston-check-${VERSION}.tar.gz` plus a sha256 sidecar published as a GitHub Release asset.
- **POSIX-sh installer** — `install.sh` published alongside the tarball. Verifies sha256 before extraction; installs to `$PREFIX/share/preston-check` with a thin shim at `$PREFIX/bin/preston-check`.
- **Homebrew tap** — `preston-check/homebrew-tap` formula auto-bumped on release when `HOMEBREW_TAP_TOKEN` secret is present.
- **Docker image** — `ghcr.io/preston-check/scan:latest` plus version tags built multi-arch (amd64 + arm64) on each release when DockerHub creds are configured.
- **GitHub Action** — `preston-check/scan-action@v1`, defined by `action.yml` at the repo root. Used as a drop-in PR security gate per `examples/github-action.yml`.

Data flow during a scan

The runner starts by parsing CLI flags, then sources each `lib/*.sh` module in order. License verification happens first because subsequent checks may need to know the effective tier. License loading reads `~/.preston-check/license` (overridable via `PRESTON_LICENSE`), extracts the PEM-style payload and signature blocks, base64-decodes both, and verifies the Ed25519 signature against the embedded public key at `lib/license_pubkey.pem`. If any of these steps fail, the tool falls back to the Free tier with a clear status note printed in the header.

Configuration loads next via the existing `load_config` function, which extracts simple `key: value` pairs from the YAML-ish config file. Language detection uses `lang/detect.sh` to find the dominant source language by file count, then loads a language profile that exposes pattern variables like `AUTH_ANNOTATION` and `BIG_DECIMAL_TYPE` for downstream checks.

OSS detection scans the source directory's LICENSE file to identify recognized OSS licenses; if found, it bumps the effective tier from Free to Pro for that scan. Branding configuration applies only at Enterprise tier; at lower tiers, brand override fields are ignored with a stderr note.

The scan loop iterates over the six check directories. For each candidate check file, the runner parses its metadata block and consults `tier_allows_check` to decide whether to execute. Checks in directories above the user's tier are silently skipped (or surfaced as `SKIP` entries in `--verbose` mode). Each executed check sources into the runner's shell, emits findings via `record`, and the runner accumulates pass/fail/warn/skip counts.

Once all checks run, the runner prints a summary and (if `--report` was passed) writes a markdown report. If `pandoc` and a PDF renderer are available, it also generates a PDF. Finally, if telemetry is opted in and airgap is off, the runner sends an anonymous score payload to the telemetry endpoint as a best-effort background task.

Open-source / commercial split

Everything in this repository is under Apache 2.0. The `lib/license.sh` module is open-source — anyone can read it, audit it, fork it. The *signing private key* and the *audit-package layer* are not in the repository and constitute the commercial moat.

The audit-package layer is what Pro and Enterprise customers actually pay for. Free customers get the markdown report. Pro customers get the report plus compliance-evidence bundling, multi-repo aggregation, and branded packaging. Enterprise customers get all of that plus white-label, auditor-ready signed PDF deliverables, custom-check authoring support, SSO, and a dedicated success contact.

Because the engine is open-source, customers can verify the privacy claim themselves: there are no hidden network calls, the only optional outbound request is the opt-in telemetry, and the license verification is fully offline. This is intentionally counterintuitive — closing the engine would weaken the trust story that makes the paid tier sellable.

Trust tiers and the safety perimeter

Community contributions go through four trust tiers tied to filesystem location. A check in `checks/community/proposed/` runs only with `--include-proposed` and is treated as untrusted until reviewed. A check in `checks/community/accepted/` has passed two maintainer reviews plus the automated lint gates. A check in `checks/community/verified/` has been promoted after at least six months of field validation and a positive core-team vote. Checks in `checks/core/` are maintainer-authored.

The runner derives `trust_tier` from path, never from declared metadata. This means a malicious contributor cannot bypass review by writing `trust_tier: verified` in the metadata block — the runner ignores that field entirely. The `tools/lint-check.sh` linter enforces additional constraints at

the directory level: community checks must use IDs in the 200-999 range, must declare `runtime_class: static-grep`, and must not contain network calls, `eval`, `exec`, or file writes outside `/tmp`. At runtime, the recommended hardening (not yet implemented) is to wrap community-tier check execution in `bwrap` or `firejail` on Linux to enforce the read-only filesystem invariant.

License signing model

A single Ed25519 keypair is generated by the operator (you) once, via `tools/setup-signing-key.sh`. The private key lives at `~/.preston-check/keys/private.pem` and never leaves your machine. The public half is committed to the repository at `lib/license_pubkey.pem` so every customer instance can verify licenses offline.

To issue a license, you run `tools/issue-license.sh` with the customer ID, tier, and expiry date. The tool builds a JSON payload, signs it with the private key, and emits a PEM-style `.license` file the customer can install at `~/.preston-check/license` or pass via the `PRESTON_LICENSE` environment variable.

Rotating the signing key invalidates every existing license, so plan rotations carefully: re-issue all licenses, then push the new public key, then publish a new release. Back up the private key to an offline secure location.

Telemetry as one of three optional outbound calls

The tool makes up to three optional network calls — all opt-in, all disabled unconditionally by `--airgap`:

1. **Telemetry** — anonymous score ping to `app.preston-check.com/api/v1/telemetry`. Triggered by `--telemetry-opt-in`, `PRESTON_TELEMETRY=1`, or `telemetry: opt_in` in the config. Payload: tool version, license tier, primary language, aggregate counts, a SHA-256 hash of the git remote origin URL (or source path), UTC timestamp. Never includes source code, file paths, file names, customer details, or specific check IDs. See `docs/telemetry.md`.
2. **AI augmentation** — finding context (file:line:content + ~10 surrounding lines) sent to Anthropic, OpenAI, or local Ollama for classification and explanation. Triggered by `--ai-augment` or `PRESTON_AI=1` plus an API key.
3. **AI auto-fix** — same scope as augmentation but with ~30 surrounding lines (15 before, 15 after) sent so the model can produce a safe unified diff. Triggered by `--ai-fix` (which implies `--ai-augment`).

All three are short, auditable, and live in dedicated lib files so the privacy story is verifiable by reading the source.

The aggregate telemetry data feeds the annual State of Fintech Security report, which is the tentpole content marketing artifact. The first edition's methodology and template ships at `docs/state-of-fintech-security/2026.md`.

Threat-intel auto-ingestion

The catalog grows automatically through a weekly GitHub Actions run of `tools/sync-threat-intel.py` (`.github/workflows/threat-intel-sync.yml`, Mondays 09:00 UTC). The pipeline pulls fintech-relevant CVEs from NIST NVD, drafts community-tier check files into `checks/community/proposed/`, persists processed-CVE state in `.preston-check/threat-intel-state.json`, and opens a PR via `peter-evans/create-pull-request@v6` when there are changes. Maintainer review converts drafts to authored grep patterns and promotes them to `checks/community/accepted/`. Designed for triage-via-Admin-Portal in the SaaS layer (see `docs/portals-and-kpis.md`).